

Reviewer Pack — Super-Exponential Lower Bound on Extended Frege Proofs for R...

Entry a8c40c3a34db · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 06:53:57 UTC

Super-Exponential Lower Bound on Extended Frege Proofs for Ramsey-Type Tautologies

Entry ID: a8c40c3a34db **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 06:53:57 UTC

1. Conjecture

Field A (mathematical branch): PROOF_SYSTEMS **Field B** (complexity object): COMBINATORIAL_PRINCIPLES

Statement:

For any Ramsey-type tautology encoding the non-existence of monochromatic cliques in colored graphs, the size of Extended Frege proofs is super-exponential in the input size.

Rationale (proposer’s reasoning):

Ramsey-type tautologies encode combinatorial principles with inherent structural complexity. By linking their proof size to the combinatorial hardness of Ramsey numbers, we hypothesize that Extended Frege cannot efficiently prove these tautologies, leveraging the interplay between proof complexity and combinatorial thresholds.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9809d58b1754165b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_ramsey_tautology(n):
        # Generate a Ramsey-type tautology for a graph with n vertices
        edges = []
        for i in range(n):
            for j in range(i + 1, n):
                if random.choice([True, False]):
                    edges.append((i, j))
        return edges

    def extended_frege_proof_size(edges):
        # Simplified model of Extended Frege proof size
        return len(edges) ** 2

    n = random.randint(5, 40)
    tautology = generate_ramsey_tautology(n)
    proof_size = extended_frege_proof_size(tautology)

    return {
        "metric_name": "proof_size",
        "metric_value": proof_size,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30)) # Default to 1

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)
```

```

mean_value = sum(res["metric_value"] for res in results) / len(results)
std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

if all(res["conjecture_holds"] for res in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not res["conjecture_holds"] for res in results):
    first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'proof_size', 'metric_value': 64, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'proof_size', 'metric_value': 22801, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'proof_size', 'metric_value': 15876, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'proof_size', 'metric_value': 196, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'proof_size', 'metric_value': 148225, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'proof_size', 'metric_value': 3364, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'proof_size', 'metric_value': 96721, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'proof_size', 'metric_value': 7744, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'proof_size', 'metric_value': 3481, 'instances_tested': 1, 'conjecture_holds': False}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e0bd3894.py", line 63, in <module>
    first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
                        ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e0bd3894.py", line 63, in <genexpr>
    first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
                        ~~~~~^~~~~~
KeyError: 'seed'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before producing reliable data, and the counterexample 'mapping_undefined' is ambiguous. | next: Fix the test to handle seed parameters properly and re-run with validated input configurations

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	90808
2	propose	ollama_remote	qwen3:8b	0	0	85621
3	preregistration	ollama_remote	qwen3:8b	0	0	27360
4	novelty	ollama_remote	qwen3:8b	0	0	24043
5	novelty	ollama_remote	qwen3:8b	0	0	16698
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39972
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	4281
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7050
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6405
10	judge	ollama_remote	qwen3:8b	0	0	20409

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 322648 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/a8c40c3a34db.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a8c40c3a34db.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a8c40c3a34db.tar.gz` (if generated)